

UNIVERSIDAD SIGLO 21

Taller de Construcción de Software — INF243

TRABAJO PRÁCTICO INTEGRADOR

EcoMarket

Plataforma de Comercio Electrónico Sostenible

Integrante:	Mauro Vera
Matrícula:	SOF01586
Materia:	Taller de Construcción de Software (INF243)
Profesor:	Pablo Martín Maldonado
Modalidad:	Individual
Entrega:	Entrega Final (Semana 2) — 29/06/2026
Período académico:	1B. 2026

Índice

1. Introducción y contexto del proyecto
2. Arquitectura del sistema
3. Modelo de datos
4. Diseño de la API REST
5. Frontend: estructura de componentes y flujo de datos
6. Integración frontend-backend y resolución de CORS
7. Testing
8. Uso de inteligencia artificial
9. Conclusiones
10. Reflexión individual

1. Introducción y contexto del proyecto

El presente informe documenta el desarrollo de EcoMarket, una plataforma de comercio electrónico para la venta de productos ecológicos y sostenibles, desarrollada como Trabajo Práctico Integrador de la materia Taller de Construcción de Software (INF243) de la Universidad Siglo 21.

El proyecto consiste en una aplicación full-stack compuesta por un backend que expone una API REST construida con Spring Boot, y un frontend desarrollado en React que consume dicha API. El objetivo funcional es permitir a un cliente de EcoMarket navegar un catálogo de productos, gestionar un carrito de compras con cálculo de totales, y confirmar pedidos que queden registrados en un historial de órdenes.

El enunciado del trabajo otorga libertad de diseño sobre la implementación concreta (nombres de entidades, estructura de componentes, organización del código), exigiendo únicamente que la funcionalidad descrita esté presente y que las decisiones tomadas estén justificadas. Este informe documenta y justifica cada una de esas decisiones.

2. Arquitectura del sistema

El sistema se organiza en dos componentes desacoplados que se comunican mediante HTTP/JSON:

- Backend: API REST construida con Spring Boot 3.5 sobre Java 21, siguiendo una arquitectura en capas (Controller → Service → Repository), con persistencia gestionada mediante Spring Data JPA / Hibernate sobre una base de datos MySQL.
- Frontend: Single Page Application construida con React 18 y Vite, compuesta exclusivamente por componentes funcionales que consumen la API mediante Axios.

2.1. Justificación del uso de REST

Se eligió una arquitectura REST por sobre alternativas como GraphQL o RPC por las siguientes razones: (1) es el estándar de la industria para este tipo de aplicaciones CRUD con recursos bien definidos (productos, carrito, órdenes); (2) permite testear cada endpoint de forma aislada con herramientas como Postman, tal como exige la consigna; (3) el bajo acoplamiento entre frontend y backend facilita que ambos evolucionen de forma independiente, comunicándose únicamente a través de contratos JSON estables.

2.2. Arquitectura en capas del backend

Se aplicó una separación estricta de responsabilidades:

- Controller: recibe las peticiones HTTP, valida el formato de entrada y delega la lógica al Service. No contiene lógica de negocio.
- Service: concentra las reglas de negocio (por ejemplo, no permitir confirmar un pedido con el carrito vacío, o calcular el total del carrito).
- Repository: abstrae el acceso a datos mediante interfaces de Spring Data JPA, sin SQL manual para las operaciones estándar.

- DTO (Data Transfer Object): se utilizan records de Java para transportar datos entre la API y el cliente, evitando exponer las entidades JPA directamente y desacoplando el modelo de persistencia del contrato público de la API.
 - Mapper: una clase dedicada (EcoMapper) centraliza la conversión entre entidades y DTOs, evitando duplicar esa lógica en los servicios.
 - Exception handling: un manejador centralizado (@RestControllerAdvice) captura las excepciones de dominio y las traduce a respuestas HTTP con códigos de estado apropiados y un formato de error uniforme.
-

3. Modelo de datos

El modelo de datos se diseñó alrededor de tres entidades principales: Producto, Carrito (con sus ítems) y Orden (con sus ítems).

3.1. Entidades

- Producto: representa un artículo del catálogo, con nombre, descripción, categoría, precio y stock disponible.
- Carrito / ItemCarrito: el carrito agrupa ítems, cada uno referenciando un producto y una cantidad. El total se calcula dinámicamente a partir de los ítems vigentes.
- Orden / ItemOrden: representa un pedido confirmado. Incluye fecha y hora de confirmación, un mensaje personalizado opcional, y la lista de ítems incluidos.

3.2. Decisión de diseño: snapshot de datos en la orden

Una decisión relevante fue que ItemOrden no referencia simplemente al Producto original, sino que almacena una copia (snapshot) de su nombre y precio al momento de confirmar el pedido. Esta decisión se tomó para garantizar la integridad del historial: si un producto es modificado o eliminado del catálogo después de haberse vendido, las órdenes pasadas deben seguir mostrando el precio y nombre con el que efectivamente se vendió, sin alterarse retroactivamente.

3.3. Carrito único

Dado que el alcance del trabajo práctico no contempla un sistema de autenticación de usuarios, se optó por modelar un carrito único para la aplicación, que se crea de forma automática ante la primera operación y se vacía al confirmarse un pedido. Esta es una simplificación consciente y declarada: en un sistema productivo con múltiples usuarios concurrentes, el carrito debería asociarse a una sesión o cuenta de usuario autenticada.

4. Diseño de la API REST

Todos los endpoints responden en formato JSON y utilizan los códigos de estado HTTP correspondientes a la operación realizada (200 para lecturas exitosas, 201 para creaciones, 204 para eliminaciones sin

contenido de retorno, 400 para errores de validación o de negocio, y 404 cuando el recurso solicitado no existe).

Recurso	Método	Endpoint	Descripción
Productos	GET	/api/productos	Listar catálogo completo
Productos	GET	/api/productos/{id}	Detalle de un producto
Productos	POST	/api/productos	Crear producto (201)
Productos	PUT	/api/productos/{id}	Modificar producto
Productos	DELETE	/api/productos/{id}	Eliminar producto (204)
Carrito	GET	/api/carrito	Ver carrito con total
Carrito	POST	/api/carrito/items	Agregar ítem (201)
Carrito	PUT	/api/carrito/items/{itemId}	Modificar cantidad
Carrito	DELETE	/api/carrito/items/{itemId}	Quitar ítem
Carrito	DELETE	/api/carrito	Vaciar carrito
Órdenes	POST	/api/ordenes	Confirmar pedido (201)
Órdenes	GET	/api/ordenes	Historial de órdenes
Órdenes	GET	/api/ordenes/{id}	Detalle de una orden

La totalidad de los endpoints fue probada con Postman antes de iniciar la integración con el frontend, siguiendo lo solicitado en la consigna. La colección exportada con ejemplos de request y response se incluye en el archivo `postman/EcoMarket.postman_collection.json`, adjunto a esta entrega.

4.1. Manejo de errores

Las excepciones de negocio (por ejemplo, intentar confirmar un pedido con el carrito vacío, o solicitar un producto con un id inexistente) son capturadas centralizadamente y traducidas a respuestas JSON con una estructura uniforme, conteniendo un mensaje descriptivo y el código de estado HTTP correspondiente. Esto evita que el cliente reciba stack traces de Java o respuestas inconsistentes entre distintos endpoints.

5. Frontend: estructura de componentes y flujo de datos

El frontend fue desarrollado con React 18 utilizando exclusivamente componentes funcionales, conforme a lo exigido por la consigna. Se hizo uso obligatorio de los hooks `useState` (para el manejo de estado local de cada componente) y `useEffect` (para disparar las llamadas a la API al montar los componentes o cuando cambian sus dependencias).

5.1. Estructura de componentes

- Navbar: navegación entre las vistas de Catálogo, Carrito e Historial.
- Catalogo / ProductoCard / ProductoForm / ProductoDetalle: listado y gestión CRUD de productos.
- Carrito: visualización de ítems, modificación de cantidades y cálculo de total.
- ConfirmarPedido: formulario para agregar el mensaje personalizado y confirmar la orden.
- HistorialOrdenes: listado de órdenes confirmadas con sus ítems y mensaje asociado.
- Componentes de soporte de UX: Modal, Toast, Cargando y AlertaError, para dar feedback visual consistente ante operaciones asíncronas y errores.

5.2. Flujo de datos unidireccional

Se respetó el patrón unidireccional propio de React: el componente App concentra el estado global relevante (principalmente el estado del carrito) y lo distribuye a los componentes hijos mediante props. Las actualizaciones originadas en los componentes hijos (por ejemplo, agregar un producto al carrito desde el catálogo) se comunican hacia arriba mediante funciones callback pasadas como props, que terminan disparando una actualización de estado en el componente padre correspondiente.

Cada vista que es responsable de un recurso particular (por ejemplo, Catálogo o Historial) solicita sus propios datos al backend mediante useEffect al montarse, en lugar de depender de que el componente padre le inyecte esa información, lo que mantiene la responsabilidad de cada componente acotada a los datos que efectivamente necesita.

5.3. Experiencia de usuario

Se incorporaron estados de carga (mostrados mientras se espera la respuesta del backend), manejo visible de errores (mediante el componente AlertaError) y confirmaciones de acciones relevantes (mediante Toast), de forma que el usuario tenga retroalimentación clara ante cada operación: agregar al carrito, confirmar un pedido, o eliminar un producto.

6. Integración frontend-backend y resolución de CORS

Dado que el frontend (servido por Vite en `http://localhost:5173`) y el backend (servido por Spring Boot en `http://localhost:8080`) se ejecutan en orígenes distintos durante el desarrollo local, fue necesario configurar explícitamente CORS (Cross-Origin Resource Sharing) en el backend para permitir que el navegador autorice las peticiones del frontend.

6.1. El problema concreto

Sin una configuración explícita de CORS, el navegador bloquea las peticiones del frontend hacia la API con un error de origen cruzado, incluso si el backend está funcionando correctamente y respondería sin problemas a una petición hecha directamente (por ejemplo, desde Postman, que no aplica las restricciones CORS del navegador).

6.2. La solución implementada

Se implementó una configuración de CORS a nivel de aplicación (CorsConfig) que habilita explícitamente el origen del frontend de desarrollo, permitiendo los métodos HTTP utilizados (GET, POST, PUT, DELETE) y las cabeceras necesarias para el envío de JSON. Esto fue verificado comprobando que las respuestas de tipo preflight (OPTIONS) del backend incluyeran la cabecera Access-Control-Allow-Origin con el valor correspondiente al origen del frontend.

6.3. Otra dificultad de integración: conflicto de puertos en MySQL

Durante las pruebas de integración en el entorno local de desarrollo se detectó que la máquina ya contaba con una instancia de MySQL escuchando en el puerto 3306 (el puerto por defecto), lo cual generaba un conflicto con el contenedor Docker definido para este proyecto y provocaba errores de autenticación al intentar conectar el backend a la base de datos equivocada.

La solución adoptada fue publicar el contenedor MySQL del proyecto en el puerto 3307 del host (manteniendo el puerto interno 3306 del contenedor sin modificar) y configurar el backend para apuntar explícitamente a ese puerto. Esta decisión evita conflictos con instalaciones preexistentes de MySQL en la máquina de desarrollo, sin requerir desinstalar ni detener servicios ajenos al proyecto.

7. Testing

Se implementó una estrategia de testing en dos niveles para el backend y uno para el frontend, totalizando 67 pruebas automatizadas, todas en estado exitoso (100%) al momento de la entrega:

Suite	Herramientas	Cantidad
Backend — pruebas unitarias	JUnit 5 + Mockito	22
Backend — pruebas de integración	MockMvc + base H2 en memoria	24
Frontend — pruebas de componentes	Vitest + React Testing Library	21

7.1. Pruebas unitarias e integración (backend)

Las pruebas unitarias verifican la lógica de cada Service de forma aislada, simulando (mockeando) las dependencias del Repository. Las pruebas de integración levantan el contexto completo de Spring Boot apuntando a una base de datos H2 en memoria, permitiendo validar el comportamiento real de los controladores end-to-end (incluyendo serialización JSON y códigos de estado HTTP) sin depender de una instancia de MySQL externa.

7.2. Pruebas de componentes (frontend)

Las pruebas de frontend, escritas con Vitest y React Testing Library, validan el comportamiento de los componentes principales (catálogo, carrito y confirmación de pedido) desde la perspectiva del usuario:

que se rendericen los datos correctos, que los clics disparen los callbacks esperados, y que los estados de carga y error se muestren apropiadamente.

7.3. Verificación de instalación limpia

Adicionalmente a las pruebas automatizadas, se realizó una verificación manual de instalación en limpio: se clonó el repositorio en un directorio nuevo, se levantó la base de datos vía Docker Compose, y se inició el backend y el frontend siguiendo únicamente las instrucciones documentadas en el README, sin pasos manuales adicionales. Se comprobó el flujo completo de uso (catálogo → carrito con cálculo de total → confirmación de pedido con mensaje personalizado → historial de órdenes) funcionando correctamente de punta a punta.

8. Uso de inteligencia artificial

Conforme a lo solicitado explícitamente en la consigna del trabajo práctico, se declara a continuación el uso de herramientas de inteligencia artificial generativa durante el desarrollo de este proyecto.

8.1. Herramientas utilizadas

- Claude Code (Anthropic): utilizado para la generación del código fuente del backend (Spring Boot) y del frontend (React), incluyendo las entidades, capas de servicio, controladores, componentes de React y las pruebas automatizadas.
- Claude (claude.ai, interfaz de chat): utilizado para la planificación de las tareas, la resolución de problemas de entorno (configuración de Docker, conflicto de puertos de MySQL, autenticación de Git con GitHub mediante token de acceso personal), la verificación del proyecto mediante una instalación en limpio, y la redacción de este informe técnico y del archivo README.md.

8.2. Partes del proyecto generadas con asistencia de IA

La totalidad del código fuente del backend y del frontend, así como las pruebas automatizadas y la colección de Postman, fueron generados con asistencia de Claude Code a partir de instrucciones detalladas que especificaban el alcance funcional completo del trabajo práctico (extraído textualmente de la consigna), los requisitos técnicos obligatorios (patrón de capas, uso de DTOs, hooks de React, flujo unidireccional de datos) y la exigencia de que el 100% de las pruebas automatizadas debían pasar exitosamente antes de continuar con cada etapa siguiente.

La documentación (README.md y el presente informe técnico) fue elaborada con asistencia de Claude a partir de los resultados reales verificados del proyecto: estructura de archivos, endpoints implementados, resultados de las pruebas, y los problemas de integración efectivamente encontrados durante el proceso (CORS y conflicto de puertos de MySQL).

8.3. Proceso de verificación

Antes de documentar el proyecto como funcional, se realizó una verificación independiente clonando el repositorio en un directorio limpio y ejecutando el flujo completo de instalación y uso sin asistencia ni intervención manual fuera de lo documentado, confirmando que el resultado generado con asistencia de IA efectivamente cumple con la funcionalidad requerida por la consigna.

9. Conclusiones

El desarrollo de EcoMarket permitió aplicar de forma integrada los conceptos centrales de la materia: arquitectura en capas, diseño de una API REST, persistencia con JPA/Hibernate, desarrollo de una interfaz con React siguiendo el paradigma de componentes funcionales y flujo de datos unidireccional, y la integración real entre ambas partes resolviendo problemas concretos de entorno como CORS.

El proyecto cumple con la totalidad del alcance funcional solicitado: gestión completa del catálogo de productos (alta, baja, modificación y consulta), un carrito de compras funcional con cálculo de totales, y un flujo de confirmación de pedidos con mensaje personalizado e historial consultable. La calidad del resultado se respalda en 67 pruebas automatizadas exitosas y en una verificación manual de instalación en limpio, que confirmó que la aplicación funciona de punta a punta siguiendo únicamente las instrucciones documentadas en el README.

Como principal aprendizaje transversal, el proceso reforzó la importancia de no asumir que algo funciona simplemente porque las pruebas automatizadas pasan: fue la verificación manual de instalación en limpio la que permitió detectar y corregir problemas reales de entorno (un conflicto de puerto de MySQL, y un push de Git que no había llegado al repositorio remoto) que las pruebas unitarias y de integración, al depender de una base de datos en memoria, no podían detectar por sí solas.

10. Reflexión individual

[BORRADOR — Mauro: ajustá este texto a tu experiencia real antes de entregar. Lo armé en base a lo que efectivamente pasó en el proceso, pero la idea es que quede en tu voz y puedas defenderlo sin problema en la presentación oral.]

¿Qué aprendí?

Más allá de los aspectos técnicos de Spring Boot y React, el principal aprendizaje de este trabajo práctico tuvo que ver con la diferencia entre que el código funcione y que el proyecto funcione. Las pruebas automatizadas (67 en total, todas exitosas) me dieron confianza en que la lógica de negocio estaba bien implementada, pero no garantizaban que alguien pudiera clonar el repositorio y levantarlo siguiendo únicamente el README. Tuve que detenerme a probar específicamente ese escenario, y ahí aparecieron dos problemas reales que las pruebas no podían detectar: un conflicto de puerto entre el MySQL de Docker del proyecto y un MySQL que ya tenía corriendo localmente en mi máquina, y un caso donde el código nunca había llegado a sincronizarse con el repositorio remoto de GitHub por un tema de autenticación.

Esto me permitió entender de forma concreta por qué, en un entorno profesional, la integración y el despliegue se tratan como una etapa con su propio proceso de verificación, distinta de escribir el código y de que las pruebas unitarias pasen.

¿Qué haría diferente?

Si rehiciera el trabajo, probaría la instalación en limpio del proyecto de forma incremental desde el principio (por ejemplo, después de cada etapa significativa) en lugar de dejarlo para el final, ya que hubiera detectado antes el problema de sincronización con el repositorio remoto, en lugar de descubrirlo recién al intentar validar todo el proyecto de punta a punta.

¿Cuál fue mi principal aporte?

Al tratarse de una entrega individual, me hice cargo de la totalidad del proceso: desde la definición del alcance técnico (modelo de datos, endpoints, estructura de componentes) hasta la verificación final del producto, pasando por la resolución de los problemas de configuración de entorno y autenticación que surgieron en el camino. El aporte que más valoro es haber insistido en validar el proyecto con una instalación en limpio antes de darlo por terminado, en lugar de confiar únicamente en el resultado reportado por las pruebas automatizadas.